

Programación Concurrente y Distribuida

Stranger Things: Simulación de Hawkins y el Upside Down

Grado en Ingeniería de Computadores

Curso 2025/2026 – Convocatoria Ordinaria

DNI: 05952615P

Apellidos, Nombre: Gálvez Esquivel, Julián

DNI: 55128253J

Apellidos, Nombre: Rodríguez Gil, Adrián

Índice

1. Análisis de alto nivel
2. Diseño general del sistema y herramientas de sincronización utilizadas
3. Descripción de las clases principales
4. Diagrama de clases
5. Anexo: código fuente

1. Análisis de alto nivel

1.1. Descripción general del problema

El objetivo de esta práctica es desarrollar una simulación concurrente y distribuida basada en la ciudad ficticia de Hawkins y el Upside Down. En el sistema existen varios tipos de entidades que actúan de forma simultánea: niños, demogorgons y eventos globales. Cada uno de estos elementos tiene un comportamiento propio y debe coordinarse correctamente con el resto para evitar inconsistencias, accesos simultáneos incorrectos a recursos compartidos o estados imposibles dentro de la simulación.

La práctica se divide en dos partes principales. La primera parte se centra en la programación concurrente, modelando los niños, los demogorgons y los eventos globales como hilos que se ejecutan de forma paralela. La segunda parte amplía el sistema mediante programación distribuida, incorporando un módulo remoto basado en RMI que permite consultar el estado del sistema en tiempo real y pausar o reanudar la ejecución desde un cliente remoto.

El escenario se divide en dos dimensiones principales:

- **Hawkins**, que representa la zona segura.
- **Upside Down**, que representa la zona peligrosa donde se encuentran los demogorgons.

Dentro de Hawkins existen tres zonas: Calle Principal, Sótano de los Byers y Radio WSQK. Dentro del Upside Down existen cuatro zonas inseguras: Bosque, Laboratorio, Centro Comercial y Alcantarillado. Además, existe una zona especial llamada Colmena, donde son trasladados los niños capturados por los demogorgons.

Los niños realizan un ciclo indefinido de exploración: parten de la Calle Principal, pasan por el Sótano de los Byers, esperan a formar grupo para cruzar un portal, entran en una zona del Upside Down, recolectan sangre de Vecna, regresan a Hawkins y acuden a Radio WSQK antes de volver a iniciar el ciclo. Por su parte, los demogorgons se mueven entre las zonas inseguras, buscan niños y tratan de capturarlos con una determinada probabilidad.

El sistema también incorpora eventos globales aleatorios que afectan a todos los hilos activos. Estos eventos son el apagón del laboratorio, la tormenta del Upside Down, la intervención de Eleven y la red mental. Cada uno de ellos modifica temporalmente el comportamiento de los niños, los demogorgons o los portales.

1.2. Actores principales del sistema

Los actores principales identificados en el sistema son los siguientes:

Niños

Los niños son entidades activas del sistema y están implementados como hilos. Cada niño tiene un identificador único con formato NXXXX. Su comportamiento consiste en desplazarse por las zonas de Hawkins y del Upside Down, cruzar portales, recolectar sangre y regresar a Hawkins. Durante su estancia en el Upside Down pueden ser atacados por un demogorgon.

Demogorgons

Los demogorgons también están implementados como hilos. Cada demogorgon tiene un identificador único con formato DXXXX. El primer demogorgon creado es el Demogorgon Alpha, con identificador D0000. Los demogorgons se mueven por las zonas del Upside Down y atacan a los niños presentes. Cada demogorgon mantiene un contador propio de capturas, que posteriormente se utiliza para generar el ranking remoto.

Mundo

La clase Mundo actúa como elemento central de coordinación. Contiene las listas compartidas de niños y demogorgons, los portales, los contadores, el control de pausa, el control de ataques y los métodos remotos. Es el recurso compartido más importante del sistema y, por tanto, la clase donde se concentra la mayor parte de la sincronización.

Portales

Los portales conectan Hawkins con cada una de las cuatro zonas inseguras del Upside Down. Cada portal tiene una capacidad distinta para formar grupos de ida. El portal del Bosque requiere grupos de 2 niños, el del Laboratorio grupos de 3, el del Centro Comercial grupos de 4 y el del Alcantarillado grupos de 2. Una vez formado

el grupo, los niños cruzan de uno en uno. Además, los niños que intentan volver a Hawkins tienen prioridad frente a los que quieren entrar al Upside Down.

Eventos globales

Los eventos globales son generados por un hilo independiente de la clase Eventos. Cada cierto tiempo se activa aleatoriamente uno de los cuatro eventos posibles. Durante su duración, modifican el comportamiento del sistema. Por ejemplo, el apagón bloquea los portales, Eleven paraliza a los demogorgons y libera niños de la Colmena, la tormenta modifica los tiempos, y la red mental dirige a los demogorgons hacia la zona con más niños.

Interfaz gráfica local

La clase Interfaz muestra gráficamente el estado completo de la simulación en el servidor. Permite visualizar las listas de niños y demogorgons en cada zona, las colas de los portales, los niños en Hawkins, la Colmena, la sangre acumulada y el evento activo.

Interfaz remota

La clase InterfazRemota implementa el cliente remoto mediante RMI. Permite consultar el número de niños en Hawkins, el número de niños en cada portal, el número de niños y demogorgons en cada zona, el ranking de demogorgons, el evento activo y el tiempo restante. También permite pausar y reanudar el programa principal.

2. Diseño general del sistema y herramientas de sincronización utilizadas

2.1. Diseño general

El sistema se ha diseñado siguiendo una estructura centralizada en la clase Mundo. Esta clase almacena el estado común de la simulación y ofrece métodos sincronizados para modificar o consultar dicho estado. Los hilos Niños, Demogorgons y Eventos no gestionan directamente las estructuras compartidas, sino que acceden a ellas a través de métodos de Mundo.

La clase principal PECLPA realiza las tareas iniciales del sistema. En primer lugar, crea una instancia de Mundo y una instancia de Eventos. Después, registra el objeto remoto mediante RMI utilizando *LocateRegistry.createRegistry(1099)* y *Naming.rebind*. A continuación, lanza la interfaz gráfica local, inicia el hilo de eventos, crea el demogorgon inicial y genera progresivamente los 1500 niños con intervalos aleatorios entre 0,5 y 2 segundos.

El comportamiento concurrente se basa en que cada niño y cada demogorgon funcionan de forma indefinida, repitiendo su ciclo continuamente. Los niños recorren las zonas de Hawkins y del Upside Down, mientras que los demogorgons buscan víctimas y se desplazan entre las zonas inseguras. El hilo de eventos se ejecuta en paralelo y modifica el comportamiento global de todos ellos.

La parte distribuida se implementa con Java RMI. La clase Mundo extiende *UnicastRemoteObject* e implementa la interfaz remota *InterfazMundo*. De esta forma, el cliente remoto puede consultar información del sistema y ejecutar operaciones de pausa o reanudación sin acceder directamente a las estructuras internas.

2.2. Gestión de zonas y entidades

Las zonas se representan mediante listas de objetos. En Mundo existen listas para los niños en cada zona del Upside Down, listas para los demogorgons en cada zona, listas para las zonas de Hawkins y una lista específica para la Colmena.

Para los niños en Hawkins y en la Colmena se emplea *CopyOnWriteArrayList*, mientras que en otras zonas se utilizan listas sincronizadas mediante *Collections.synchronizedList*. Esta elección permite trabajar con estructuras seguras frente a accesos concurrentes en un contexto en el que varios hilos pueden entrar, salir o consultar simultáneamente las mismas zonas.

Los métodos *entrarNiño*, *salirNiño*, *entrarDemogorgon* y *eliminarListaDemogorgon* centralizan los cambios de ubicación. Estos métodos están sincronizados para evitar modificaciones simultáneas inconsistentes sobre las listas compartidas.

Además, el método *eliminarNiñoDeTodasLasListas* se utiliza para asegurar que un niño no quede duplicado en varias zonas cuando es capturado, liberado o reinicia su ciclo.

2.3. Sincronización mediante *synchronized*

El mecanismo de sincronización más utilizado es *synchronized*. Se emplea en varios niveles:

- En métodos de Niños, para proteger variables internas como *capturado*, *siendoAtacado* y *liberadoPorEleven*.
- En métodos de Mundo, para proteger listas compartidas y contadores.
- En métodos de Eventos, para proteger el estado de los eventos globales.
- En métodos remotos de Mundo, para devolver valores consistentes al cliente remoto.

El uso de *synchronized* permite que solo un hilo a la vez ejecute determinadas secciones críticas, evitando problemas como condiciones de carrera, lecturas inconsistentes o modificaciones simultáneas sobre los mismos datos.

Por ejemplo, el estado de un niño durante un ataque debe modificarse de forma segura. Para ello, los métodos *intentarSerAtacado*, *finalizarAtaque*, *liberarPorEleven*, *fueLiberadoPorEleven* y *resetLiberadoPorEleven* están sincronizados. Esto evita que dos demogorgons puedan atacar al mismo niño al mismo tiempo o que un niño sea liberado y capturado de forma inconsistente.

2.4. Sincronización de portales con *wait()* y *notifyAll()*

La parte más delicada de la práctica es el control de los portales. Cada portal se representa mediante una clase interna llamada *Portal*, que contiene:

- *capacidad*: número de niños necesarios para formar el grupo de ida.
- *listaEsperalda*: cola de niños esperando para entrar al Upside Down.
- *listaEsperaVuelta*: cola de niños esperando para volver a Hawkins.
- *grupoldaActual*: grupo de niños que ya ha sido formado para cruzar.
- *cruzando*: indica si hay un niño cruzando el portal.
- *grupoFormado*: indica si ya existe un grupo exclusivo formado.

El método *esperarEnPortal* gestiona la entrada al Upside Down. Cuando un niño llega al portal, se añade a la lista de espera de ida. Si no hay apagón, no hay nadie cruzando, no hay niños esperando para volver y se alcanza el tamaño necesario del grupo, se forma un grupo exclusivo. A partir de ese momento, los niños de ese grupo cruzan de uno en uno. Los niños que lleguen después no pueden incorporarse a ese grupo, sino que deben esperar al siguiente.

El método *volverDePortal* gestiona la vuelta a Hawkins. En este caso no se necesita formar grupo. El niño solo debe esperar si hay apagón o si otro niño está cruzando. Además, la vuelta tiene prioridad sobre la ida, ya que el método de ida comprueba que *listaEsperaVuelta* esté vacía antes de permitir el cruce hacia el Upside Down.

La sincronización se realiza con *synchronized (p)*, siendo *p* el portal concreto. Dentro de ese bloque se usan *wait()* y *notifyAll()* para dormir y despertar hilos cuando cambian las condiciones del portal.

Este diseño permite cumplir las condiciones principales del enunciado:

- Cada portal forma grupos del tamaño correspondiente.
- Los grupos de ida son exclusivos.
- Los niños cruzan de uno en uno.
- Los niños que vuelven a Hawkins tienen prioridad.
- Durante un apagón no se permite iniciar nuevos cruces.
- Los niños que ya estaban cruzando terminan su cruce normalmente.

2.5. Uso de *ReentrantLock* en los ataques

Para controlar los ataques de los demogorgons se utiliza un *ReentrantLock* llamado *atacar*. Este bloqueo protege la selección de objetivos y la fase final del ataque.

Cuando un demogorgon intenta atacar, primero comprueba si hay niños en la zona. Si los hay, selecciona uno aleatoriamente. Antes de iniciar el ataque, llama al método *intentarSerAtacado* del niño. Este método comprueba si el niño ya está capturado o si ya está siendo atacado. Si no lo está, marca al niño como *siendoAtacado*.

El uso del *ReentrantLock* evita que varios demogorgons seleccionen y capturen simultáneamente al mismo niño. Una vez transcurrido el tiempo del ataque, se decide si el niño resiste o si es capturado. En caso de captura, se elimina de todas las listas, se añade a la Colmena, se incrementa el contador de capturas del demogorgon y se comprueba si debe generarse un nuevo demogorgon.

2.6. Control de pausa y reanudación

El sistema incorpora una funcionalidad de pausa y reanudación tanto local como remota. Para ello, la clase *Mundo* contiene:

```
private final Object lockPausa = new Object();  
private volatile boolean pausado = false;
```

El atributo pausado se declara como *volatile* para asegurar que los cambios realizados por un hilo sean visibles para el resto. El método *comprobarPausa* utiliza *wait()* sobre el objeto *lockPausa*. Cuando el programa está pausado, los hilos quedan bloqueados hasta que se llama a reanudar.

El método pausar activa la pausa estableciendo *pausado = true*. El método reanudar establece *pausado = false* y despierta a los hilos con *notifyAll()*.

Además, se han creado métodos de espera como *esperar* y *esperarConPausa*, que dividen las esperas largas en intervalos pequeños. Esto permite que la pausa sea más sensible, ya que los hilos no quedan bloqueados durante varios segundos completos sin comprobar si el sistema ha sido pausado.

2.7. Eventos globales

La clase Eventos se ejecuta como un hilo independiente. Cada 30-60 segundos activa un evento aleatorio con una duración de entre 5 y 10 segundos. Los eventos disponibles son:

Apagón del laboratorio

Durante el apagón, los portales quedan inutilizables. Los métodos de los portales comprueban si hay apagón antes de permitir nuevos cruces. Cuando el apagón termina, se llama a *despertarPortales*, que hace *notifyAll()* sobre todos los portales para que los niños bloqueados puedan volver a comprobar sus condiciones.

Tormenta del Upside Down

Durante la tormenta, el tiempo de recolección de los niños se duplica. Además, los tiempos de ataque o espera de los demogorgons se reducen, haciendo que su comportamiento sea más agresivo.

Intervención de Eleven

Durante la intervención de Eleven, los demogorgons quedan paralizados. Esto se implementa haciendo que los demogorgons permanezcan en bucles de espera mientras *evento.hayEleven()* sea verdadero. Al terminar el evento, Eleven libera de la Colmena tantos niños como unidades de sangre se hayan recolectado durante dicho evento.

Red mental

Durante la red mental, los demogorgons dejan de moverse aleatoriamente y se dirigen a la zona del Upside Down con mayor número de niños. Para ello se utiliza el método *zonaConMasNiños*.

2.8. Programación distribuida mediante RMI

La parte distribuida se implementa utilizando Java RMI. Para ello se define la interfaz *InterfazMundo*, que extiende *Remote* y declara los métodos que podrán ser invocados desde el cliente remoto.

La clase *Mundo* implementa esta interfaz y extiende *UnicastRemoteObject*, por lo que puede ser registrada como objeto remoto. En la clase principal *PECLPA*, el objeto se registra con:

```
LocateRegistry.createRegistry(1099);  
Naming.rebind("//localhost/MundoStranger", mundo);
```

El cliente remoto, implementado en *InterfazRemota*, obtiene la referencia al objeto remoto mediante:

```
mundoRemoto = (InterfazMundo)  
Naming.lookup("rmi://localhost:1099/MundoStranger");
```

A partir de ese momento, la interfaz remota puede consultar periódicamente el estado del sistema. Para ello se utiliza un hilo que llama a *actualizarRemoto()* cada segundo. La actualización de los componentes Swing se realiza mediante *SwingUtilities.invokeLater*, respetando el modelo de ejecución de Swing.

La interfaz remota permite consultar:

- Número de niños en Hawkins.
- Número de niños en cada portal.
- Número de niños en cada zona del Upside Down.
- Número de niños capturados en la Colmena.
- Número de demogorgons en cada zona.
- Ranking de los tres demogorgons con más capturas.
- Evento global activo.
- Tiempo restante del evento.
- Pausar y reanudar la simulación.

2.9. Sistema de log

La clase *LogHawkins* implementa un sistema de log en el fichero hawkins.txt. El método escribir está declarado como *synchronized static*, lo que asegura que solo un hilo pueda escribir en el fichero al mismo tiempo.

Cada línea del log incluye una marca temporal con fecha y hora, seguida del mensaje correspondiente. Este recurso compartido resulta importante, ya que todos los niños, demogorgons y eventos pueden escribir en el fichero simultáneamente.

3. Descripción de las clases principales

3.1. Clase *PECLPA*

La clase *PECLPA* contiene el método *main* y actúa como punto de entrada del programa servidor.

Responsabilidades principales

- Crear el objeto compartido Mundo.
- Crear e iniciar el hilo de eventos globales.
- Registrar el objeto remoto mediante RMI.
- Crear y mostrar la interfaz gráfica local.
- Crear el demogorgon inicial.
- Crear progresivamente los 1500 niños.

Funcionamiento

Primero se instancia Mundo y se crea el objeto Eventos, asociándolo al mundo mediante *setEventos*. Después se crea el registro RMI en el puerto 1099 y se registra el objeto remoto con el nombre MundoStranger.

La interfaz local se lanza mediante *SwingUtilities.invokeLater*. Además, se crea un hilo de refresco que actualiza la interfaz cuatro veces por segundo, aproximadamente cada 250 milisegundos.

Finalmente, se inicia el hilo de eventos, se crea el demogorgon inicial D0000 y se generan 1500 niños en intervalos aleatorios de entre 0,5 y 2 segundos.

3.2. Clase *Niños*

La clase *Niños* extiende *Thread*, por lo que cada niño se ejecuta de forma independiente.

Atributos principales

- *mundo*: referencia al objeto compartido Mundo.
- *eventos*: referencia al gestor de eventos globales.
- *id*: identificador del niño con formato NXXXX.
- *capturado*: indica si el niño está capturado.
- *siendoAtacado*: indica si el niño está siendo atacado.
- *liberadoPorEleven*: indica si el niño ha sido liberado por Eleven.

Métodos principales

- *intentarSerAtacado()*: comprueba si el niño puede ser atacado.
- *finalizarAtaque(boolean capturado)*: actualiza el estado del niño tras un ataque.
- *liberarPorEleven()*: libera al niño de la Colmena.
- *fueLiberadoPorEleven()*: consulta si fue liberado.
- *resetLiberadoPorEleven()*: reinicia el indicador de liberación.
- *esperar(int milisegundos)*: realiza esperas comprobando periódicamente la pausa.
- *run()*: contiene el ciclo de vida del niño.

Ciclo de vida

El método *run* implementa un ciclo indefinido. En cada iteración, el niño entra en Calle Principal, espera entre 3 y 5 segundos, pasa al Sótano de los Byers, espera entre 1 y 2 segundos, elige una zona aleatoria del Upside Down y espera en el portal correspondiente.

Una vez cruza el portal, entra en la zona elegida y recolecta sangre. Si hay tormenta, el tiempo de recolección se duplica. Durante este proceso puede ser atacado por un demogorgon. Si es capturado, espera hasta ser liberado por Eleven. Si sobrevive, vuelve a Hawkins por el portal, entrega la sangre y se dirige a Radio WSQK. Después del descanso, vuelve a comenzar el ciclo.

3.3. Clase *Mundo*

La clase *Mundo* es la clase central de la aplicación. Extiende *UnicastRemoteObject* e implementa *InterfazMundo*, por lo que actúa tanto como recurso compartido local como objeto remoto RMI.

Atributos principales

Contiene listas para representar las zonas:

- *niñosBosque*
- *niñosLaboratorio*
- *niñosCentroComercial*
- *niñosAlcantarillado*
- *niñosCallePrincipal*
- *niñosSotanoByers*
- *niñosRadioWSQK*
- *niñosColmena*

También contiene listas para los demogorgons:

- *demogorgonsBosque*
- *demogorgonsLaboratorio*
- *demogorgonsCentroComercial*
- *demogorgonsAlcantarillado*
- *todosDemogorgons*

Además, contiene contadores y mecanismos de sincronización:

- *niñosEnColmena*
- *contadorDemogorgons*
- *contadorSangre*
- *contadorSangreDuranteEleven*
- *atacar*
- *lockPausa*
- *pausado*

Métodos principales

- *esperarEnPortal(int zona, Niños n)*: controla el cruce de ida al Upside Down.
- *volverDePortal(int zona, Niños n)*: controla la vuelta a Hawkins.
- *despertarPortales()*: despierta a los hilos esperando en los portales.
- *entrarNiño(int zona, Niños n)*: añade un niño a una zona.

- *salirNiño(int zona, Niños n)*: elimina un niño de una zona.
- *eliminarNiñoDeTodasLasListas(Niños n)*: elimina un niño de todas las zonas.
- *incrementarSangre()*: incrementa la sangre recolectada.
- *revivirNiños()*: libera niños de la Colmena durante la intervención de Eleven.
- *demogorgonAtacar(int num, Demogorgons d)*: implementa la lógica de ataque.
- *zonaConMasNiños()*: devuelve la zona con mayor número de niños.
- *comprobarPausa()*: bloquea hilos si el sistema está pausado.
- *pausar()*: pausa el sistema.
- *reanudar()*: reanuda el sistema.

Métodos remotos

La clase también implementa métodos remotos como:

- *getHawkinsRemoto()*
- *getPortal1Remoto()*
- *getNiñosBosqueRemoto()*
- *getDemogorgonsBosqueRemoto()*
- *getRankingDemogorgonsRemoto()*
- *getEventoActualRemoto()*
- *getTiempoEventoRemoto()*
- *pausarRemoto()*
- *reanudarRemoto()*

Estos métodos permiten que la interfaz remota consulte el estado del sistema y controle la pausa/reanudación.

3.4. Clase interna *Portal*

La clase *Portal* se encuentra dentro de *Mundo* y representa cada uno de los portales.

Atributos principales

- *capacidad*: tamaño necesario del grupo.
- *listaEsperalda*: niños esperando para entrar al Upside Down.
- *listaEsperaVuelta*: niños esperando para volver a Hawkins.
- *grupoldaActual*: grupo de ida ya formado.
- *cruzando*: indica si hay un niño cruzando.

- *grupoFormado*: indica si hay un grupo activo.

Función

Esta clase permite encapsular la información necesaria para gestionar cada portal de manera independiente. Cada portal tiene su propio monitor de sincronización, lo que permite que los portales funcionen de manera paralela entre sí.

3.5. Clase *Demogorgons*

La clase *Demogorgons* extiende *Thread*, por lo que cada demogorgon se ejecuta de forma concurrente.

Atributos principales

- *mundo*: referencia al objeto compartido.
- *evento*: referencia al gestor de eventos.
- *id*: identificador con formato DXXXX.
- *zona*: zona actual del demogorgon.
- *zona_anterior*: zona anterior.
- *capturas*: número de niños capturados.

Métodos principales

- *moverse()*: mueve al demogorgon a una nueva zona.
- *incrementar_capturas()*: incrementa su contador de capturas.
- *getCapturas()*: devuelve el número de capturas.
- *esperar(int milisegundos)*: realiza esperas comprobando la pausa.
- *run()*: contiene el ciclo de vida del demogorgon.

Ciclo de vida

El demogorgon se registra en el sistema y entra en un ciclo indefinido. En cada iteración comprueba si el sistema está pausado y si Eleven está activa. Si no hay apagón, abandona su zona anterior y se mueve a una nueva. Si está activa la red mental, se dirige a la zona con más niños; en caso contrario, elige una zona aleatoria distinta de la anterior.

Después de moverse, llama a *mundo.demogorgonAtacar*, que contiene la lógica de selección de víctima, duración del ataque, probabilidad de captura y traslado a la Colmena.

3.6. Clase *Eventos*

La clase *Eventos* extiende *Thread* y controla la aparición de eventos globales.

Atributos principales

- *mundo*: referencia al mundo.
- *num*: evento seleccionado.
- *apagon*: indica si hay apagón.
- *tormenta*: indica si hay tormenta.
- *eleven*: indica si Eleven está activa.
- *redMental*: indica si está activa la red mental.
- *finEventoMillis*: instante estimado de finalización del evento.

Métodos principales

- *activarEvento(int num)*: activa el evento correspondiente.
- *desactivarEvento(int num)*: desactiva el evento correspondiente.
- *elegirEvento()*: selecciona y ejecuta un evento aleatorio.
- *getEventoActual()*: devuelve el nombre del evento activo.
- *getTiempoRestanteEvento()*: devuelve el tiempo restante del evento.
- *run()*: genera eventos periódicamente.

Funcionamiento

El hilo espera un tiempo aleatorio entre 30 y 60 segundos, activa un evento aleatorio durante 5-10 segundos y después lo desactiva. Si el evento era la intervención de Eleven, al terminar se llama a *mundo.revivirNiños()*.

3.7. Clase *LogHawkins*

La clase *LogHawkins* se encarga de escribir los eventos del sistema en el fichero *hawkins.txt*.

Atributos principales

- *ARCHIVO*: nombre del fichero de log.
- *formato*: formato de fecha y hora.

Método principal

- *escribir(String mensaje)*: escribe una línea en el fichero con marca temporal.

El método está declarado como *synchronized static*, lo que garantiza que las escrituras concurrentes no se mezclen entre sí.

3.8. Interfaz *InterfazMundo*

La interfaz *InterfazMundo* define los métodos remotos accesibles desde el cliente RMI. Extiende *Remote*, y todos sus métodos lanzan *RemoteException*.

Función

Permite separar la implementación interna de Mundo de las operaciones visibles para el cliente remoto. De esta forma, la interfaz remota no accede directamente a las listas ni a los atributos internos, sino únicamente a métodos controlados.

Métodos incluidos

Incluye métodos para consultar niños, portales, demogorgons, evento activo, tiempo restante, ranking y operaciones de pausa/reanudación.

3.9. Clase *Interfaz*

La clase *Interfaz* representa la interfaz gráfica local del servidor. Extiende *JFrame*.

Función

Muestra el estado detallado del sistema en tiempo real. Incluye áreas de texto para las zonas de Hawkins, los portales, las zonas del Upside Down, los demogorgons, la Colmena, la sangre acumulada y el evento activo.

Método principal

- *actualizar()*: obtiene la información de Mundo y actualiza los componentes gráficos.

La actualización se realiza mediante *SwingUtilities.invokeLater* para asegurar que los cambios en la interfaz se hagan correctamente en el hilo de eventos de *Swing*.

3.10. Clase *InterfazRemota*

La clase *InterfazRemota* representa el cliente remoto. También extiende *JFrame*.

Función

Se conecta al servidor mediante RMI y muestra información resumida del sistema. Además, permite pausar y reanudar la simulación de forma remota.

Funcionamiento

En el constructor realiza el *Naming.lookup* del objeto remoto *MundoStranger*. Después lanza un hilo que llama periódicamente a *actualizarRemoto*.

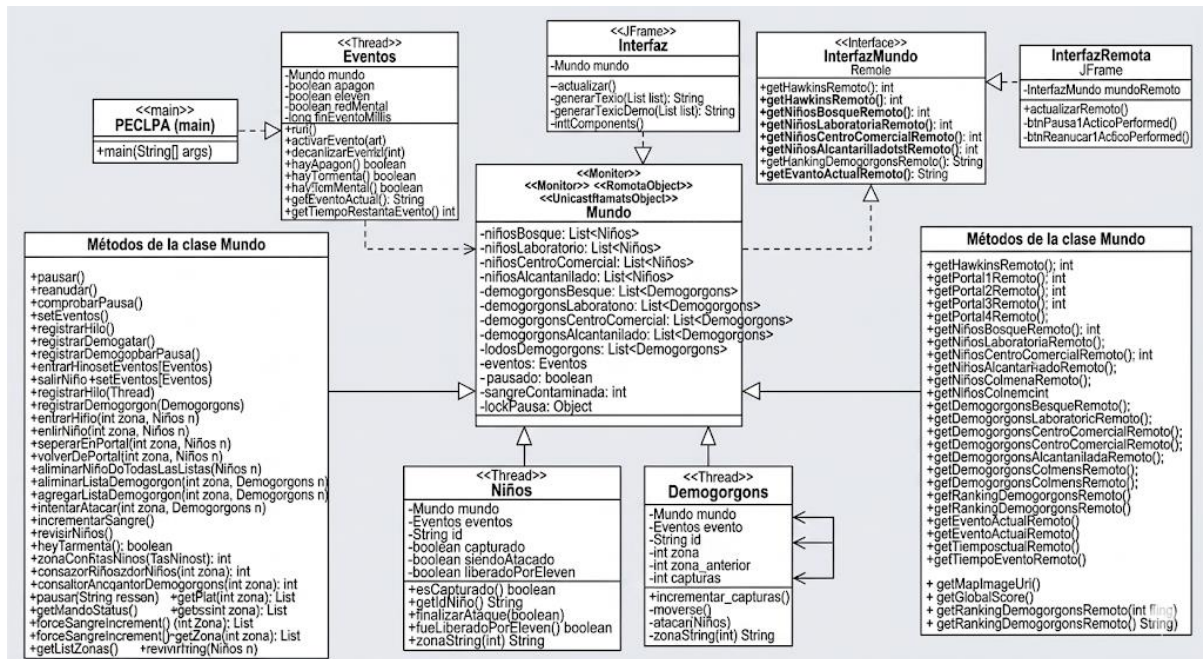
Método principal

- *actualizarRemoto()*: invoca métodos remotos sobre *mundoRemoto* y actualiza los campos de la interfaz.

Los botones *PAUSAR* y *REANUDAR* llaman respectivamente a *pausarRemoto()* y *reanudarRemoto()*.

4. Diagrama de clases

El siguiente diagrama representa de forma simplificada las relaciones principales del sistema:



4.1. Relaciones principales

- PECLPA crea e inicia Mundo, Eventos, Interfaz, Demogorgons y Niños.
- Niños y Demogorgons acceden a Mundo para modificar su posición y consultar el estado.
- Eventos modifica estados globales que afectan a niños, demogorgons y portales.
- Mundo implementa InterfazMundo para permitir acceso remoto.
- InterfazRemota accede a Mundo mediante RMI.
- Interfaz accede directamente a Mundo porque forma parte del programa servidor.

5. Anexo: código fuente

En este apartado se incluye el código fuente completo del proyecto.

El código se ha organizado en las siguientes clases:

- *PECLPA.java*

- *Niños.java*
- *Mundo.java*
- *LogHawkins.java*
- *InterfazRemota.java*
- *InterfazMundo.java*
- *Interfaz.java*
- *Eventos.java*
- *Demogorgons.java*

PECLPA.java:

```

15 public class PECLPA {
16
17     public static void main(String[] args) {
18         try {
19             Mundo mundo = new Mundo();
20             Eventos eventos = new Eventos(mundo);
21             mundo.setEventos(eventos);
22
23             LocateRegistry.createRegistry(1099);
24             Naming.rebind("//localhost/MundoStranger", mundo);
25
26             javax.swing.SwingUtilities.invokeLater(() -> {
27                 try {
28
29                     Interfaz interfaz = new Interfaz(mundo);
30                     interfaz.setVisible(true);
31
32                     Thread hiloRefresco = new Thread(() -> {
33                         while (true) {
34                             try {
35                                 javax.swing.SwingUtilities.invokeLater(() -> {
36                                     interfaz.actualizar();
37                                 });
38
39                                 Thread.sleep(250); // 4 veces por segundo es suficiente para que se vea fluido
40                             } catch (Exception e) {
41                                 System.err.println("Error en el hilo de refresco: " + e.getMessage());
42                             }
43                         }
44                     });
45
46                     hiloRefresco.setDaemon(true);
47                     hiloRefresco.start();
48
49                 } catch (Exception e) {
50                     System.err.println("No se pudo iniciar la interfaz: " + e.getMessage());
51                 }
52             });
53
54             eventos.start();
55
56             Demogorgons d = new Demogorgons(mundo, eventos, 0);
57             d.start();
58
59             for (int i = 0; i < 1500; i++) {
60                 Niños n = new Niños(mundo, eventos, i);
61                 n.start();
62
63                 try {
64                     Thread.sleep((int) (Math.random() * 1500) + 500);
65                 } catch (Exception e) {
66                     System.err.println("Error: " + e.getMessage());
67                 }
68             }
69
70         } catch (Exception e) {
71             System.err.println("Error: " + e.getMessage());
72         }
73     }
74 }
75
76 }
77

```

Niños.java:

```

10 public class Niños extends Thread {
11     private Mundo mundo;
12     private Eventos eventos;
13     private String id;
14     private boolean capturado = false;
15     private boolean siendoAtacado = false;
16     private boolean liberadoPorEleven = false;
17
18     public Niños(Mundo mundo, Eventos eventos, int numid) {
19         this.mundo = mundo;
20         this.eventos = eventos;
21         this.id = String.format("N%04d", numid);
22         LogHawkins.escribir("Niño " + id + " creado");
23     }
24
25     public synchronized void setCapturado(boolean estado) {
26         this.capturado = estado;
27     }
28
29     public synchronized boolean esCapturado() {
30         return capturado;
31     }
32
33     public String getIdNiño() {
34         return id;
35     }
36
37     private String zonaString(int zona) {
38         switch (zona) {
39             case 1:
40                 return "El Bosque";
41             case 2:
42                 return "El Laboratorio";
43             case 3:
44                 return "El Centro Comercial";
45             case 4:
46                 return "El Alcantarillado";
47
48             default:
49                 return null;
50         }
51     }
52

```

```

53 public synchronized boolean intentarSerAtacado() {
54     if (capturado || siendoAtacado) {
55         LogHawkins.escribir("Niño " + id + " está siendo atacado");
56         return false;
57     }
58     siendoAtacado = true;
59     return true;
60 }
61
62
63 public synchronized void finalizarAtaque(boolean capturado) {
64     this.capturado = capturado;
65     this.siendoAtacado = false;
66     if(capturado){
67         LogHawkins.escribir("Niño " + id + " ha sido capturado");
68     }else{
69         LogHawkins.escribir("Niño " + id + " ha sobrevivido al ataque");
70     }
71 }
72
73 public synchronized void liberarPorEleven() {
74     capturado = false;
75     siendoAtacado = false;
76     liberadoPorEleven = true;
77     LogHawkins.escribir("Niño " + id + " ha sido liberado por Eleven");
78 }
79
80 public synchronized boolean fueLiberadoPorEleven() {
81     return liberadoPorEleven;
82 }
83
84 public synchronized void resetLiberadoPorEleven() {
85     liberadoPorEleven = false;
86 }
87
88 private void esperar(int milisegundos) throws InterruptedException {
89     int tiempoTranscurrido = 0;
90     while (tiempoTranscurrido < milisegundos) {
91         Thread.sleep(100);
92         tiempoTranscurrido += 100;
93         mundo.comprobarPausa();
94     }
95 }

```

```

98  @Override
99
100  public String toString() {
101      return id;
102  }
103
104  public void run() {
105      mundo.registrarHilo(this);
106      LogHawkins.escribir("Niño " + id + " inicia ejecución");
107      while (true) {
108          mundo.comprobarPausa();
109          try {
110              mundo.eliminarNiñoDeTodasLasListas(this);
111              mundo.entrarNiño(5, this);
112              LogHawkins.escribir("Niño " + id + " entra en Calle Principal");
113
114              esperar((int) (Math.random() * 2000) + 3000);
115
116              mundo.comprobarPausa();
117              mundo.salirNiño(5, this);
118              mundo.entrarNiño(6, this);
119              LogHawkins.escribir("Niño " + id + " entra en el Sótano de los Byers");
120
121              esperar((int) (Math.random() * 1000) + 1000);
122
123              mundo.comprobarPausa();
124
125              int zonaElegida = (int) (Math.random() * 4) + 1;
126              mundo.salirNiño(6, this);
127
128              mundo.esperarEnPortal(zonaElegida, this);
129              LogHawkins.escribir("Niño " + id + " cruza al Upside Down hacia " + zonaString(zonaElegida));
130
131              mundo.comprobarPausa();
132
133              while(esCapturado()){
134                  try {
135                      esperar(200);
136                      mundo.comprobarPausa();
137                  } catch (Exception e) {
138                      System.err.println("Error: " + e.getMessage());
139                  }
140              }

```

```

141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197

```

```

    if (fueLiberadoPorEleven()) {
        resetLiberadoPorEleven();
        continue;
    }

    mundo.entrarNiño(zonaElegida, this);

    int tiempoRecolectando = (int) (Math.random() * 2000) + 3000;
    if(mundo.hayTormenta()){
        esperar(tiempoRecolectando * 2);
    } else {
        esperar(tiempoRecolectando);
    }

    while(esCapturado()){
        try {
            esperar(200);
        } catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }
    }

    if (fueLiberadoPorEleven()) {
        resetLiberadoPorEleven();
        continue;
    }

    mundo.comprobarPausa();
    mundo.salirNiño(zonaElegida, this);
    mundo.volverDePortal(zonaElegida, this);

    mundo.comprobarPausa();
    mundo.incrementarSangre();
    mundo.entrarNiño(7, this);
    LogHawkins.escribir("Niño " + id + " entra en Radio WSQK");

    esperar((int) (Math.random() * 2000) + 2000);

    while(esCapturado()){
        try {
            esperar(200);
        } catch (Exception e){
            System.err.println("Error: " + e.getMessage());
        }
    }

    if (fueLiberadoPorEleven()) {
        resetLiberadoPorEleven();
        continue;
    }

    mundo.salirNiño(7, this);
} catch (Exception e) {
    System.err.println("Error: " + e.getMessage());
}
}
}
}

```

Mundo.java:

```
22 public class Mundo extends UnicastRemoteObject implements InterfazMundo{
23     private List<Niños> niñosBosque = Collections.synchronizedList(new ArrayList<>());
24     private List<Niños> niñosLaboratorio = Collections.synchronizedList(new ArrayList<>());
25     private List<Niños> niñosCentroComercial = Collections.synchronizedList(new ArrayList<>());
26     private List<Niños> niñosAlcantarillado = Collections.synchronizedList(new ArrayList<>());
27     private List<Demogorgons> demogorgonsBosque = Collections.synchronizedList(new ArrayList<>());
28     private List<Demogorgons> demogorgonsLaboratorio = Collections.synchronizedList(new ArrayList<>());
29     private List<Demogorgons> demogorgonsCentroComercial = Collections.synchronizedList(new ArrayList<>());
30     private List<Demogorgons> demogorgonsAlcantarillado = Collections.synchronizedList(new ArrayList<>());
31     private List<Niños> niñosCallePrincipal = new CopyOnWriteArrayList<>();
32     private List<Niños> niñosColmena = new CopyOnWriteArrayList<>();
33     private List<Niños> niñosSotanoByers = Collections.synchronizedList(new ArrayList<>());
34     private List<Niños> niñosRadioWSQK = Collections.synchronizedList(new ArrayList<>());
35     private List<Thread> hilosActivos = Collections.synchronizedList(new ArrayList<>());
36     private List<Demogorgons> todosDemogorgons = Collections.synchronizedList(new ArrayList<>());
37
38     private int niñosEnColmena=0;
39     private int contadorDemogorgons=1;
40     private int contadorSangre=0;
41     private int contadorSangreDuranteEleven = 0;
42
43     private Lock atacar = new ReentrantLock();
44     private Eventos eventos;
45
46
47     private final Object lockPausa = new Object();
48     private volatile boolean pausado = false;
49
50
51     public void setEventos(Eventos eventos) {
52         this.eventos = eventos;
53     }
54
55
56     private class Portal {
57         int capacidad;
58
59         List<Niños> listaEsperaIda = Collections.synchronizedList(new ArrayList<>());
60         List<Niños> listaEsperaVuelta = Collections.synchronizedList(new ArrayList<>());
61
62         List<Niños> grupoIdaActual = new ArrayList<>();
63
64
65         int cruzando = 0;
66         boolean grupoFormado = false;
67     }
68
69     private Portal bosque = new Portal();
70     private Portal laboratorio = new Portal();
71     private Portal centro = new Portal();
72     private Portal alcantarillado = new Portal();
73
74
75     public Mundo() throws RemoteException{
76         bosque.capacidad = 2;
77         laboratorio.capacidad = 3;
78         centro.capacidad = 4;
79         alcantarillado.capacidad = 2;
80     }
81
82
83
84     private Portal getPortal(int zona) {
85         return switch (zona) {
86             case 1 -> bosque;
87             case 2 -> laboratorio;
88             case 3 -> centro;
89             case 4 -> alcantarillado;
90             default -> throw new IllegalArgumentException();
91         };
92     }
93
94     private String nombreZona(int zona) {
95         return switch (zona) {
96             case 1 -> "BOSQUE";
97             case 2 -> "LABORATORIO";
98             case 3 -> "CENTRO COMERCIAL";
99             case 4 -> "ALCANTARILLADO";
100             default -> "DESCONOCIDO";
101         };
102     }
103 }
```

```

104 public void esperarEnPortal(int zona, Niños n) {
105     comprobarPausa();
106
107     Portal p = getPortal(zona);
108     String destino = nombreZona(zona);
109
110     synchronized (p) {
111         p.listaEsperaIda.add(n);
112         System.out.println("Niño " + n.getIdNiño() + " espera portal hacia " + destino);
113
114         while (true) {
115
116             // Si todavía no hay un grupo formado, se intenta formar uno.
117             if (!p.grupoFormado
118                 && !eventos.hayApagon()
119                 && p.cruzando == 0
120                 && p.listaEsperaVuelta.isEmpty()
121                 && p.listaEsperaIda.size() >= p.capacidad) {
122
123                 p.grupoFormado = true;
124                 p.grupoIdaActual.clear();
125
126                 for (int i = 0; i < p.capacidad; i++) {
127                     Niños niñoGrupo = p.listaEsperaIda.remove(0);
128                     p.grupoIdaActual.add(niñoGrupo);
129                 }
130
131                 p.notifyAll();
132             }
133
134             /*
135             * El niño solo puede cruzar si:
136             * Ya pertenece al grupo actual.
137             * Es el primero de ese grupo.
138             * Nadie está cruzando el portal.
139             * No hay apagón.
140             * No hay niños esperando para volver, porque la vuelta tiene prioridad.
141             */
142             if (p.grupoIdaActual.contains(n)
143                 && p.grupoIdaActual.get(0) == n
144                 && p.cruzando == 0
145                 && !eventos.hayApagon()
146                 && p.listaEsperaVuelta.isEmpty()) {
147

```

```

148         p.cruzando = 1;
149         p.grupoIdaActual.remove(0);
150         break;
151     }
152
153     try {
154         p.wait();
155     } catch (InterruptedException e) {
156         Thread.currentThread().interrupt();
157         return;
158     }
159 }
160
161
162 System.out.println("Niño " + n.getIdNiño() + " cruza hacia " + destino);
163 LogHawkins.escribir("El niño " + n.getIdNiño() + " ha cruzado el portal hacia " + destino);
164
165 esperarConPausa(1000);
166
167 synchronized (p) {
168     p.cruzando = 0;
169
170     // Cuando ya han cruzado todos los niños del grupo, el portal queda libre
171     // para formar un nuevo grupo.
172     if (p.grupoIdaActual.isEmpty()) {
173         p.grupoFormado = false;
174     }
175
176     p.notifyAll();
177 }
178
179
180 public void volverDePortal(int zona, Niños n) {
181     comprobarPausa();
182
183     Portal p = getPortal(zona);
184     String origen = nombreZona(zona);
185
186     synchronized (p) {
187         p.listaEsperaVuelta.add(n);
188         System.out.println("Niño " + n.getIdNiño() + " espera para volver desde " + origen);
189         p.notifyAll();
190
191         /*
192          * Solo espera si hay apagón o si otro niño está cruzando.
193          */
194         while (eventos.hayApagon() || p.cruzando > 0) {
195             try {
196                 p.wait();
197             } catch (InterruptedException e) {
198                 Thread.currentThread().interrupt();
199                 return;
200             }
201         }
202
203         p.listaEsperaVuelta.remove(n);
204         p.cruzando = 1;
205     }
206
207     System.out.println("Niño " + n.getIdNiño() + " REGRESA desde " + origen);
208     LogHawkins.escribir("El niño " + n.getIdNiño() + " ha regresado a Hawkins desde " + origen);
209
210     esperarConPausa(1000);
211
212     synchronized (p) {
213         p.cruzando = 0;
214         p.notifyAll();
215     }
216 }
217
218
219 public void despertarPortales() {
220     synchronized (bosque) { bosque.notifyAll(); }
221     synchronized (laboratorio) { laboratorio.notifyAll(); }
222     synchronized (centro) { centro.notifyAll(); }
223     synchronized (alcantarillado) { alcantarillado.notifyAll(); }
224 }

```

```

226 public synchronized void entrarNiño(int zona, Niños n){
227     switch(zona){
228     case 1:
229         niñosBosque.add(n);
230         break;
231     case 2:
232         niñosLaboratorio.add(n);
233         break;
234     case 3:
235         niñosCentroComercial.add(n);
236         break;
237     case 4:
238         niñosAlcantarillado.add(n);
239         break;
240     case 5:
241         niñosCallePrincipal.add(n);
242         break;
243     case 6:
244         niñosSotanoByers.add(n);
245         break;
246     case 7:
247         niñosRadioWSQK.add(n);
248         break;
249     case 8:
250         niñosColmena.add(n);
251         break;
252     default:
253         break;
254     }
255 }

257 public synchronized void salirNiño(int zona, Niños n) {
258     switch (zona) {
259     case 1: niñosBosque.remove(n); break;
260     case 2: niñosLaboratorio.remove(n); break;
261     case 3: niñosCentroComercial.remove(n); break;
262     case 4: niñosAlcantarillado.remove(n); break;
263     case 5: niñosCallePrincipal.remove(n); break;
264     case 6: niñosSotanoByers.remove(n); break;
265     case 7: niñosRadioWSQK.remove(n); break;
266     }
267 }

269 public synchronized void incrementarSangre(){
270     contadorSangre++;
271     if(eventos.hayEleven()){
272         contadorSangreDuranteEleven++;
273     }
274 }

276 public synchronized void entrarDemogorgon(int zona, Demogorgons d){
277     switch(zona){
278     case 1:
279         demogorgonsBosque.add(d);
280         break;
281     case 2:
282         demogorgonsLaboratorio.add(d);
283         break;
284     case 3:
285         demogorgonsCentroComercial.add(d);
286         break;
287     case 4:
288         demogorgonsAlcantarillado.add(d);
289         break;
290     default:
291         break;
292     }
293 }

```

```

296 public synchronized void eliminarListaDemogorgon(int zona, Demogorgons d) {
297     switch (zona) {
298         case 1:
299             demogorgonsBosque.remove(d);
300             break;
301         case 2:
302             demogorgonsLaboratorio.remove(d);
303             break;
304         case 3:
305             demogorgonsCentroComercial.remove(d);
306             break;
307         case 4:
308             demogorgonsAlcantarillado.remove(d);
309             break;
310     }
311 }
312
313
314 public synchronized boolean hay_niño(int num) {
315     switch (num) {
316         case 1:
317             return niñosBosque.size() >= 1;
318         case 2:
319             return niñosLaboratorio.size() >= 1;
320         case 3:
321             return niñosCentroComercial.size() >= 1;
322         case 4:
323             return niñosAlcantarillado.size() >= 1;
324         default:
325             return false;
326     }
327 }
328
329 public synchronized void eliminarNiñoDeTodasLasListas(Niños n) {
330     niñosBosque.remove(n);
331     niñosLaboratorio.remove(n);
332     niñosCentroComercial.remove(n);
333     niñosAlcantarillado.remove(n);
334     niñosCallePrincipal.remove(n);
335     niñosSotanoByers.remove(n);
336     niñosRadioWSQK.remove(n);
337     niñosColmena.remove(n);
338 }
339
340
341 public boolean hayTormenta() {
342     return eventos.hayTormenta();
343 }
344
345 public int getSangreAcumulada() {
346     return contadorSangre;
347 }
348
349
350 public synchronized int getContadorSangreDuranteEleven() {
351     return contadorSangreDuranteEleven;
352 }
353
354
355 public synchronized void revivirNiños() {
356     int cantidad = Math.min(contadorSangreDuranteEleven, niñosColmena.size());
357
358     for (int i = 0; i < cantidad; i++) {
359         Niños niñoRevivido = niñosColmena.remove(0);
360
361         niñosEnColmena--;
362
363         niñoRevivido.liberarPorEleven();
364         entrarNiño(5, niñoRevivido);
365
366         System.out.println("Eleven libera a " + niñoRevivido.getIdNiño()
367             + " y vuelve a la Calle Principal.");
368         LogHawkins.escribir("EVENTO GLOBAL: Eleven ha liberado al niño " + niñoRevivido.getIdNiño() + " de la Colmena");
369     }
370
371     contadorSangreDuranteEleven = 0;
372 }

```

```

374 private void esperarConPausa(int milisegundos) {
375     int tiempoTranscurrido = 0;
376
377     while (tiempoTranscurrido < milisegundos) {
378         comprobarPausa();
379
380         try {
381             Thread.sleep(100);
382         } catch (InterruptedException e) {
383             Thread.currentThread().interrupt();
384             return;
385         }
386
387         tiempoTranscurrido += 100;
388     }
389 }
390
391
392 public void demogorgonAtacar(int num, Demogorgons d){
393     int probabilidad = (int)(Math.random() * 3) + 1;
394     Niños objetivo = null;
395     boolean capturado=false;
396
397     while(eventos.hayEleven()){
398         try {
399             esperarConPausa(500);
400         } catch (Exception e) {
401             System.err.println("Error: " + e.getMessage());
402         }
403     }
404     atacar.lock();
405     try{
406         if (hay_niño(num)) {
407             List<Niños> lista = switch (num) {
408                 case 1 -> niñosBosque;
409                 case 2 -> niñosLaboratorio;
410                 case 3 -> niñosCentroComercial;
411                 case 4 -> niñosAlcantarillado;
412                 default -> null;
413             };
414
415             if (lista != null && !lista.isEmpty()) {
416                 Niños candidato = lista.get((int)(Math.random() * lista.size()));
417                 if (candidato.intentarSerAtacado()) {
418                     objetivo = candidato;
419                 }

```

```

420     }
421 }
422 } catch (Exception e) {
423     System.err.println("Error: " + e.getMessage());
424 } finally {
425     atacar.unlock();
426 }
427 while (eventos.hayEleven()) {
428     try {
429         esperarConPausa(500);
430     } catch (Exception e) {
431         System.err.println("Error: " + e.getMessage());
432     }
433 }
434
435 if (objetivo == null) {
436     try {
437         if (eventos.hayTormenta()) {
438             esperarConPausa((int) (Math.random() * 1000) + 4000) / 2;
439         } else {
440             esperarConPausa((int) (Math.random() * 1000) + 4000);
441         }
442     } catch (Exception e) {
443         System.err.println("Error: " + e.getMessage());
444     }
445     return;
446 }
447
448 while (eventos.hayEleven()) {
449     try {
450         esperarConPausa(500);
451     } catch (Exception e) {
452         System.err.println("Error: " + e.getMessage());
453     }
454 }
455
456 try {
457     if (eventos.hayTormenta()) {
458         esperarConPausa((int) (Math.random() * 1000) + 500) / 2;
459     } else {
460         esperarConPausa((int) (Math.random() * 1000) + 500);
461     }
462 } catch (Exception e) {
463     System.err.println("Error: " + e.getMessage());
464 }
465
466 while (eventos.hayEleven()) {
467     try {
468         esperarConPausa(500);
469     } catch (Exception e) {
470         System.err.println("Error: " + e.getMessage());
471     }
472 }
473 atacar.lock();
474 try {
475     if (probabilidad == 3) {
476         capturado = true;
477         eliminarNiñoDeTodasLasListas(objetivo);
478         niñosColmena.add(objetivo);
479         niñosEnColmena++;
480         d.incrementar_capturas();
481
482         LogHawkins.escribir("El demogorgon " + d.toString() + " ataca al niño " + objetivo.getIdNiño());
483         LogHawkins.escribir("El niño " + objetivo.getIdNiño() + " ha sido capturado");
484
485         if (niñosEnColmena % 8 == 0) {
486             new Demogorgons(this, eventos, contadorDemogorgons++).start();
487             LogHawkins.escribir("La Red Mental se expande: Un nuevo Demogorgon ha nacido.");
488         }
489     }
490
491     objetivo.finalizarAtaque(true);
492 } else {
493     objetivo.finalizarAtaque(false);
494 }
495 } catch (Exception e) {
496     System.err.println("Error: " + e.getMessage());
497 } finally {
498     atacar.unlock();

```

```

499     }
500     while(eventos.hayEleven()){
501         try {
502             esperarConPausa(500);
503         } catch (Exception e) {
504             System.err.println("Error: " + e.getMessage());
505         }
506     }
507     if(capturado){
508         try {
509             if(eventos.hayTormenta()){
510                 esperarConPausa(((int) (Math.random() * 500) + 500)/2);
511             }else{
512                 esperarConPausa((int) (Math.random() * 500) + 500);
513             }
514         } catch (Exception e) {
515             System.err.println("Error: " + e.getMessage());
516         }
517     }
518 }
519
520
521
522 public synchronized int zonaConMasNiños() {
523     int max = niñosBosque.size();
524     int zona = 1;
525
526     if (niñosLaboratorio.size() > max) {
527         max = niñosLaboratorio.size();
528         zona = 2;
529     }
530     if (niñosCentroComercial.size() > max) {
531         max = niñosCentroComercial.size();
532         zona = 3;
533     }
534     if (niñosAlcantarillado.size() > max) {
535         zona = 4;
536     }
537
538     return zona;
539 }

```

```
541 public List<Niños> getNiñosColmena() {
542     return niñosColmena;
543 }
544
545 public List<Niños> getNiñosCallePrincipal() {
546     return niñosCallePrincipal;
547 }
548
549 public List<Niños> getNiñosSotanoByers(){
550     return niñosSotanoByers;
551 }
552 public List<Niños> getNiñosRadioWSQK(){
553     return niñosRadioWSQK;
554 }
555
556 public synchronized List<Niños> getNiñosColmenaPrueba() {
557     return new ArrayList<>(niñosColmena);
558 }
559
560 public synchronized List<Niños> getNiñosCallePrincipalPrueba() {
561     return new ArrayList<>(niñosCallePrincipal);
562 }
563
564 public synchronized List<Niños> getNiñosSotanoByersPrueba(){
565     return new ArrayList<>(niñosSotanoByers);
566 }
567
568 public synchronized List<Niños> getNiñosRadioWSQKPrueba(){
569     return new ArrayList<>(niñosRadioWSQK);
570 }
571
572 public synchronized List<Niños> getEntidadesBosque() {
573     return new ArrayList<>(niñosBosque);
574 }
575
576 public synchronized List<Niños> getEntidadesLab() {
577     return new ArrayList<>(niñosLaboratorio);
578 }
579
580 public synchronized List<Niños> getEntidadesCentroComercial() {
581     return new ArrayList<>(niñosCentroComercial);
582 }
```

```

584 public synchronized List<Niños> getEntidadesAlcantarillado() {
585     return new ArrayList<> (niñosAlcantarillado);
586 }
587
588 public List<Demogorgons> getDemosBosque() {
589     return new ArrayList<> (demogorgonsBosque);
590 }
591
592 public List<Demogorgons> getDemosLab() {
593     return new ArrayList<> (demogorgonsLaboratorio);
594 }
595
596 public List<Niños> getColaEntradaPortal(int zona) {
597     Portal p = getPortal(zona);
598     synchronized (p) {
599         return new ArrayList<> (p.listaEsperaIda);
600     }
601 }
602
603 public List<Niños> getColaVolverPortal(int zona) {
604     Portal p = getPortal(zona);
605     synchronized (p) {
606         return new ArrayList<> (p.listaEsperaVuelta);
607     }
608 }
609
610 public List<Demogorgons> getDemosCentroComercial() {
611     return new ArrayList<> (demogorgonsCentroComercial);
612 }
613
614 public List<Demogorgons> getDemosAlcantarillado() {
615     return new ArrayList<> (demogorgonsAlcantarillado);
616 }
617
618 public String getEventoActual() {
619     if (eventos == null) {
620         return "SIN EVENTO ACTIVO";
621     }
622     return eventos.getEventoActual();
623 }

```

```

625 public void comprobarPausa() {
626     synchronized (lockPausa) {
627         while (pausado) {
628             try {
629                 lockPausa.wait();
630             } catch (InterruptedException e) {
631                 Thread.currentThread().interrupt();
632             }
633         }
634     }
635 }
636
637 public void pausar() {
638     pausado = true;
639 }
640
641 public void reanudar() {
642     synchronized (lockPausa) {
643         pausado = false;
644         lockPausa.notifyAll();
645     }
646 }
647
648
649
650 public void registrarHilo(Thread t) {
651     hilosActivos.add(t);
652 }
653
654
655 public synchronized int contadorNiñosHawkins(){
656     return niñosCallePrincipal.size()+niñosSotanoByers.size()+niñosRadioWSQK.size();
657 }
658
659 public synchronized int contadorNiñosdUPD(int zona){
660     int contador = 0;
661     switch (zona) {
662         case 1: contador = niñosBosque.size(); break;
663         case 2: contador = niñosLaboratorio.size(); break;
664         case 3: contador = niñosCentroComercial.size(); break;
665         case 4: contador = niñosAlcantarillado.size(); break;
666     }
667     return contador;

```

```

668 public synchronized int contadorDemogorgons(int zona){
669     int contador = 0;
670     switch (zona) {
671         case 1: contador = demogorgonsBosque.size(); break;
672         case 2: contador = demogorgonsLaboratorio.size(); break;
673         case 3: contador = demogorgonsCentroComercial.size(); break;
674         case 4: contador = demogorgonsAlcantarillado.size(); break;
675     }
676     return contador;
677 }
678
679 @Override
680 public synchronized String getEventoActualRemoto() throws RemoteException{
681     return getEventoActual();
682 }
683
684 @Override
685 public synchronized int getHawkinsRemoto() throws RemoteException{
686     return contadorNiñosHawkins();
687 }
688
689 @Override
690 public synchronized int getPortal1Remoto() throws RemoteException{
691     return bosque.listaEsperaIda.size()+bosque.listaEsperaVuelta.size()+bosque.cruzando;
692 }
693 @Override
694 public synchronized int getPortal2Remoto() throws RemoteException{
695     return laboratorio.listaEsperaIda.size()+laboratorio.listaEsperaVuelta.size()+laboratorio.cruzando;
696 }
697 @Override
698 public synchronized int getPortal3Remoto() throws RemoteException{
699     return centro.listaEsperaIda.size()+centro.listaEsperaVuelta.size()+centro.cruzando;
700 }
701 @Override
702 public synchronized int getPortal4Remoto() throws RemoteException{
703     return alcantarillado.listaEsperaIda.size()+alcantarillado.listaEsperaVuelta.size()+alcantarillado.cruzando;
704 }
705
706 @Override
707 public synchronized int getNiñosBosqueRemoto() throws RemoteException{
708     return contadorNiñosUPD(1);
709 }

```

```

710     @Override
711     public synchronized int getNiñosLaboratorioRemoto() throws RemoteException{
712         return contadorNiñosdUPD(2);
713     }
714     @Override
715     public synchronized int getNiñosCentroComercialRemoto() throws RemoteException{
716         return contadorNiñosdUPD(3);
717     }
718     @Override
719     public synchronized int getNiñosAlcantarilladoRemoto() throws RemoteException{
720         return contadorNiñosdUPD(4);
721     }
722     @Override
723     public synchronized int getNiñosColmenaRemoto() throws RemoteException{
724         return niñosEnColmena;
725     }
726     @Override
727     public synchronized int getDemogorgonsBosqueRemoto() throws RemoteException{
728         return contadorDemogorgons(1);
729     }
730     @Override
731     public synchronized int getDemogorgonsLaboratorioRemoto() throws RemoteException{
732         return contadorDemogorgons(2);
733     }
734     @Override
735     public synchronized int getDemogorgonsCentroComercialRemoto() throws RemoteException{
736         return contadorDemogorgons(3);
737     }
738     @Override
739     public synchronized int getDemogorgonsAlcantarilladoRemoto() throws RemoteException{
740         return contadorDemogorgons(4);
741     }
742     @Override
743     public synchronized int getTiempoEventoRemoto() throws RemoteException {
744         if (eventos == null) {
745             return 0;
746         }
747
748         return eventos.getTiempoRestanteEvento();
749     }

```

```

751 public synchronized void registrarDemogorgon(Demogorgons d) {
752     if (!todosDemogorgons.contains(d)) {
753         todosDemogorgons.add(d);
754     }
755 }
756
757 @Override
758 public synchronized String getRankingDemogorgonsRemoto() throws RemoteException {
759     List<Demogorgons> copia = new ArrayList<>(todosDemogorgons);
760
761     copia.sort((dl, d2) -> Integer.compare(d2.getCapturas(), dl.getCapturas()));
762
763     StringBuilder sb = new StringBuilder();
764
765     int limite = Math.min(3, copia.size());
766
767     for (int i = 0; i < limite; i++) {
768         Demogorgons d = copia.get(i);
769         sb.append(i + 1)
770           .append(". ")
771           .append(d.toString())
772           .append(" (")
773           .append(d.getCapturas())
774           .append(") ")
775           .append("\n");
776     }
777
778     if (sb.length() == 0) {
779         return "Sin demogorgons";
780     }
781
782     return sb.toString();
783 }
784
785 @Override
786 public void pausarRemoto() throws RemoteException {
787     pausar();
788 }
789 @Override
790 public void reanudarRemoto() throws RemoteException {
791     reanudar();
792 }
793 }

```

LogHawkins.java:

```

16 public class LogHawkins {
17     private static final String ARCHIVO = "hawkins.txt";
18
19     private static final DateTimeFormatter formato =
20         DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
21
22     public synchronized static void escribir(String mensaje) {
23
24         String tiempo = LocalDateTime.now().format(formato);
25
26         try (PrintWriter pw = new PrintWriter(new FileWriter(ARCHIVO, true))) {
27
28             pw.println(tiempo + " - " + mensaje);
29
30         } catch (IOException e) {
31             System.out.println("Error escribiendo log");
32         }
33     }
34 }

```

InterfazRemota.java:

```
15 public class InterfazRemota extends javax.swing.JFrame {
16
17     private static final java.util.logging.Logger logger = java.util.logging.Logger.getLogger(InterfazRemota.class.getName());
18     private InterfazMundo mundoRemoto;
19
20     /**
21      * Creates new form InterfazRemota
22      */
23     public InterfazRemota() {
24         initComponents();
25         this.getContentPane().setBackground(new java.awt.Color(45, 55, 45));
26
27         btnReanudar1.setEnabled(false);
28         btnPausa1.setEnabled(true);
29
30         try {
31             mundoRemoto = (InterfazMundo) Naming.lookup("rmi://localhost:1099/MundoStranger");
32
33             Thread hiloActualizar = new Thread(() -> {
34                 while (true) {
35                     try {
36                         actualizarRemoto();
37                         Thread.sleep(1000);
38                     } catch (Exception e) {
39                         System.err.println("Error actualizando interfaz remota: " + e.getMessage());
40                     }
41                 }
42             });
43
44             hiloActualizar.setDaemon(true);
45             hiloActualizar.start();
46
47         } catch (Exception e) {
48             e.printStackTrace();
49         }
50     }
51
52     void actualizarRemoto() {
53         try {
54             int hawkins = mundoRemoto.getHawkinsRemoto();
55
56             int nBosque = mundoRemoto.getNiñosBosqueRemoto();
57             int nLab = mundoRemoto.getNiñosLaboratorioRemoto();
58             int nCentro = mundoRemoto.getNiñosCentroComercialRemoto();
59             int nAlc = mundoRemoto.getNiñosAlcantarilladoRemoto();
60             int nColmena = mundoRemoto.getNiñosColmenaRemoto();
61
62             int dBosque = mundoRemoto.getDemogorgonsBosqueRemoto();
63             int dLab = mundoRemoto.getDemogorgonsLaboratorioRemoto();
64             int dCentro = mundoRemoto.getDemogorgonsCentroComercialRemoto();
65             int dAlc = mundoRemoto.getDemogorgonsAlcantarilladoRemoto();
66
67             int portal1 = mundoRemoto.getPortal1Remoto();
68             int portal2 = mundoRemoto.getPortal2Remoto();
69             int portal3 = mundoRemoto.getPortal3Remoto();
70             int portal4 = mundoRemoto.getPortal4Remoto();
71
72             String evento = mundoRemoto.getEventoActualRemoto();
73             int tiempoEvento = mundoRemoto.getTiempoEventoRemoto();
74
75             SwingUtilities.invokeLater(() -> {
76                 niñosHawkins.txt.setText(String.valueOf(hawkins));
77
78                 niñosBosque.txt.setText(String.valueOf(nBosque));
79                 niñosLaboratorio.txt.setText(String.valueOf(nLab));
80                 niñosCentroComercial.txt.setText(String.valueOf(nCentro));
81                 niñosAlcantarillado.txt.setText(String.valueOf(nAlc));
82                 niñosColmena.txt.setText(String.valueOf(nColmena));
83
84                 demogorgonsBosque.txt.setText(String.valueOf(dBosque));
85                 demogorgonsLaboratorio.txt.setText(String.valueOf(dLab));
86                 demogorgonsCentroComercial.txt.setText(String.valueOf(dCentro));
87                 demogorgonsAlcantarillado.txt.setText(String.valueOf(dAlc));
88
89                 portal1.txt.setText(String.valueOf(portal1));
90                 portal2.txt.setText(String.valueOf(portal2));
91                 portal3.txt.setText(String.valueOf(portal3));
92                 portal4.txt.setText(String.valueOf(portal4));
93
94                 try {
95                     ranking.txt.setText(mundoRemoto.getRankingDemogorgonsRemoto());
96                 } catch (RemoteException ex) {
97                     System.getLogger(InterfazRemota.class.getName()).log(System.Logger.Level.ERROR, (String) null, ex);
98                 }
99             });
100         }
101     }
102 }
```

```

100         eventotxt.setText(evento);
101         tiempoEventotxt.setText(tiempoEvento + " s");
102     });
103
104     } catch (Exception e) {
105         System.err.println("Error RMI: " + e.getMessage());
106     }
107 }
108
109 /**
110  * This method is called from within the constructor to initialize the form.
111  * WARNING: Do NOT modify this code. The content of this method is always
112  * regenerated by the Form Editor.
113  */
114 @SuppressWarnings("unchecked")
115 Generated Code
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141 private void btnReanudarActionPerformed(java.awt.event.ActionEvent evt) {
142     // TODO add your handling code here:
143     btnPausar.setEnabled(true);
144     btnReanudar.setEnabled(false);
145     try {
146         mundoRemoto.reanudarRemoto();
147     } catch (RemoteException ex) {
148         System.getLogger(InterfazRemota.class.getName()).log(System.Logger.Level.ERROR, (String) null, ex);
149     }
150 }
151
152 private void btnPausarActionPerformed(java.awt.event.ActionEvent evt) {
153     btnPausar.setEnabled(false); // Desactivar pausa si ya está pausado
154     btnReanudar.setEnabled(true);
155     try {
156         mundoRemoto.pausarRemoto();
157     } catch (RemoteException ex) {
158         System.getLogger(InterfazRemota.class.getName()).log(System.Logger.Level.ERROR, (String) null, ex);
159     }
160 }
161
162
163
164
165
166
167
168
169
170
171
172
173
174 public static void main(String args[]) {
175     /* Set the Nimbus look and feel */
176     //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
177     /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
178      * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
179      */
180     try {
181         for (javax.swing.UIManager.LookAndFeelInfo info : javax.swing.UIManager.getInstalledLookAndFeels()) {
182             if ("Nimbus".equals(info.getName())) {
183                 javax.swing.UIManager.setLookAndFeel(info.getClassName());
184                 break;
185             }
186         }
187     } catch (ReflectiveOperationException | javax.swing.UnsupportedLookAndFeelException ex) {
188         logger.log(java.util.logging.Level.SEVERE, null, ex);
189     }
190     //</editor-fold>
191
192     /* Create and display the form */
193     java.awt.EventQueue.invokeLater(() -> new InterfazRemota().setVisible(true));
194 }

```

InterfazMundo.java:

```
1 public interface InterfazMundo extends Remote{
2     int getHawkinsRemoto() throws RemoteException;
16
3     int getPortal1Remoto() throws RemoteException;
4     int getPortal2Remoto() throws RemoteException;
5     int getPortal3Remoto() throws RemoteException;
6     int getPortal4Remoto() throws RemoteException;
21
7     int getNiñosBosqueRemoto() throws RemoteException;
8     int getNiñosLaboratorioRemoto() throws RemoteException;
9     int getNiñosCentroComercialRemoto() throws RemoteException;
10    int getNiñosAlcantarilladoRemoto() throws RemoteException;
11    int getNiñosColmenaRemoto() throws RemoteException;
27
12    int getDemogorgonsBosqueRemoto() throws RemoteException;
13    int getDemogorgonsLaboratorioRemoto() throws RemoteException;
14    int getDemogorgonsCentroComercialRemoto() throws RemoteException;
15    int getDemogorgonsAlcantarilladoRemoto() throws RemoteException;
32
16    String getRankingDemogorgonsRemoto() throws RemoteException;
17    String getEventoActualRemoto() throws RemoteException;
18    int getTiempoEventoRemoto() throws RemoteException;
36
19    void pausarRemoto() throws RemoteException;
20    void reanudarRemoto() throws RemoteException;
39
40 }
```

Interfaz.java:

```
15 public class Interfaz extends javax.swing.JFrame {
16     private final Mundo mundo;
17     /**
18      * Creates new form Interfaz
19      */
20     public Interfaz(Mundo mundo) {
21         this.mundo = mundo;
22         initComponents();
23         this.getContentPane().setBackground(new java.awt.Color(45, 55, 45));
24     }
25
26
27     public void actualizar() {
28         try {
29             // 1. Obtención de listas de Portales (Colas de espera)
30             // Usamos generarTexto con los nuevos métodos que devuelven List<Niños>
31             String p1In = generarTexto(mundo.getColaEntradaPortal(1));
32             String p2In = generarTexto(mundo.getColaEntradaPortal(2));
33             String p3In = generarTexto(mundo.getColaEntradaPortal(3));
34             String p4In = generarTexto(mundo.getColaEntradaPortal(4));
35
36             String p1Out = generarTexto(mundo.getColaVolverPortal(1));
37             String p2Out = generarTexto(mundo.getColaVolverPortal(2));
38             String p3Out = generarTexto(mundo.getColaVolverPortal(3));
39             String p4Out = generarTexto(mundo.getColaVolverPortal(4));
40
41             // 2. Obtención de datos de Entidades (Niños y Demos) en Upside Down
42             String bosqueN = generarTexto(mundo.getEntidadesBosque());
43             String labN = generarTexto(mundo.getEntidadesLab());
44             String ccN = generarTexto(mundo.getEntidadesCentroComercial());
45             String alcanN = generarTexto(mundo.getEntidadesAlcantarillado());
46
47             String bosqueD = generarTexto(mundo.getDemosBosque());
48             String labD = generarTexto(mundo.getDemosLab());
49             String ccD = generarTexto(mundo.getDemosCentroComercial());
50             String alcanD = generarTexto(mundo.getDemosAlcantarillado());
51
52             // 3. Obtención de datos de Hawkins
53             String calleP = generarTexto(mundo.getNiñosCallePrincipalPrueba());
54             String colmena = generarTexto(mundo.getNiñosColmenaPrueba());
55             String sotano = generarTexto(mundo.getNiñosSotanoByersPrueba());
56             String radio = generarTexto(mundo.getNiñosRadioWSQKPrueba());
57         }
```

```

58 String eventoActual = mundo.getEventoActual();
59 String sangre = String.valueOf(mundo.getSangreAcumulada());
60
61 // 4. Actualización efectiva en el EDT
62 SwingUtilities.invokeLater(() -> {
63     // Portales Ida (Listas de espera)
64     txtPortal1.setText(p1In);
65     txtPortal2.setText(p2In);
66     txtPortal3.setText(p3In);
67     txtPortal4.setText(p4In);
68
69     // Portales Vuelta (Listas de espera)
70     txtVuelta1.setText(p1Out);
71     txtVuelta2.setText(p2Out);
72     txtVuelta3.setText(p3Out);
73     txtVuelta4.setText(p4Out);
74
75     // Niños Upside Down
76     txtBosque.setText(bosqueN);
77     txtLaboratorio.setText(labN);
78     txtCentroComercial.setText(ccN);
79     txtAlcantarillado.setText(alcanN);
80
81     // Demos Upside Down
82     txtBosqueDemo.setText(bosqueD);
83     txtLaboratorioDemo.setText(labD);
84     txtCentroComercialDemo.setText(ccD);
85     txtAlcantarilladoDemo.setText(alcanD);
86
87     // Hawkins
88     txtCallePrincipal.setText(calleP);
89     txtColmenaLista.setText(colmena);
90     txtSotano.setText(sotano);
91     txtRadio.setText(radio);
92
93     txtEventoActual.setText(eventoActual);
94
95     txtSangre.setText(sangre);
96 });
97
98 } catch (Exception e) {
99     System.err.println("Error en el refresco de la interfaz: " + e.getMessage());
100 }
101 }

```

```

103 // Método sencillo para no repetir el bucle for 4 veces
104 private String generarTexto(List<?> lista) {
105     if (lista == null || lista.isEmpty()) {
106         return "";
107     }
108     StringBuilder sb = new StringBuilder();
109     for (Object obj : lista) {
110         // obj.toString() llamará al método toString de Niño o de Demogorgon
111         sb.append(obj.toString()).append("\n");
112     }
113     return sb.toString();
114 }
115
116 private String generarTextoFiltrado(List<?> lista, char inicial) {
117     if (lista == null || lista.isEmpty()) return "";
118     StringBuilder sb = new StringBuilder();
119     for (Object obj : lista) {
120         String id = obj.toString();
121         if (!id.isEmpty() && id.toUpperCase().charAt(0) == inicial) {
122             sb.append(id).append("\n");
123         }
124     }
125     return sb.toString();
126 }

```

```

131 @SuppressWarnings("unchecked")
132 Generated Code

```

```

655
656 private void btnReanudarActionPerformed(java.awt.event.ActionEvent evt) {
657     // TODO add your handling code here:
658     mundo.reanudar();
659     System.out.println("Juego Reanudado");
660 }

```

```

661
662 private void btnPausaActionPerformed(java.awt.event.ActionEvent evt) {
663     // TODO add your handling code here:
664     mundo.pausar();
665     System.out.println("Juego Pausado");
666 }

```

```

131 @SuppressWarnings("unchecked")
132 Generated Code

```

```

655
656 private void btnReanudarActionPerformed(java.awt.event.ActionEvent evt) {
657     // TODO add your handling code here:
658     mundo.reanudar();
659     System.out.println("Juego Reanudado");
660 }

```

```

661
662 private void btnPausaActionPerformed(java.awt.event.ActionEvent evt) {
663     // TODO add your handling code here:
664     mundo.pausar();
665     System.out.println("Juego Pausado");
666 }

```

Eventos.java:

```

11 public class Eventos extends Thread {
12     private Mundo mundo;
13     private int num = 0;
14     private boolean apagon = false;
15     private boolean tormenta = false;
16     private boolean eleven = false;
17     private boolean redMental = false;
18     private long finEventoMillis = 0;
19
20     public Eventos(Mundo mundo) {
21         this.mundo = mundo;
22     }
23
24     public synchronized boolean hayApagon() {
25         return apagon;
26     }
27
28     public synchronized boolean hayTormenta() {
29         return tormenta;
30     }
31
32     public synchronized boolean hayEleven() {
33         return eleven;
34     }
35
36     public synchronized boolean hayRedMental() {
37         return redMental;
38     }
39
40     public synchronized void activarEvento(int num) {
41         switch (num) {
42             case 0:
43                 apagon = true;
44                 System.out.println("EVENTO: APAGON");
45                 LogHawkins.escribir("EVENTO GLOBAL: APAGON DEL LABORATORIO iniciado");
46                 break;
47
48             case 1:
49                 tormenta = true;
50                 System.out.println("EVENTO: TORMENTA");
51                 LogHawkins.escribir("EVENTO GLOBAL: TORMENTA DEL UPSIDE DOWN iniciada");
52                 break;

```

```

54     case 2:
55         eleven = true;
56         System.out.println("EVENTO: ELEVEN");
57         LogHawkins.escribir("EVENTO GLOBAL: INTERVENCION DE ELEVEN iniciada");
58         break;
59
60     case 3:
61         redMental = true;
62         System.out.println("EVENTO: RED MENTAL");
63         LogHawkins.escribir("EVENTO GLOBAL: LA RED MENTAL iniciada");
64         break;
65     }
66 }
67
68 public synchronized void desactivarEvento(int num) {
69     switch(num) {
70     case 0:
71         apagon = false;
72         mundo.despertarPortales();
73         LogHawkins.escribir("EVENTO GLOBAL: APAGON DEL LABORATORIO finalizado");
74         break;
75
76     case 1:
77         tormenta = false;
78         LogHawkins.escribir("EVENTO GLOBAL: TORMENTA DEL UPSIDE DOWN finalizada");
79         break;
80
81     case 2:
82         eleven = false;
83         LogHawkins.escribir("EVENTO GLOBAL: INTERVENCION DE ELEVEN finalizada");
84         break;
85
86     case 3:
87         redMental = false;
88         LogHawkins.escribir("EVENTO GLOBAL: LA RED MENTAL finalizada");
89         break;
90     }
91
92     System.out.println("EVENTO FINALIZADO");
93 }

```

```

95 public void elegirEvento() {
96     num = (int) (Math.random()*4);
97
98     int duracion = (int) (Math.random()*5000) + 5000;
99
100     synchronized (this) {
101         finEventoMillis = System.currentTimeMillis() + duracion;
102     }
103
104     activarEvento(num);
105
106     try {
107         Thread.sleep(duracion);
108     } catch (Exception e) {
109         System.err.println("Error: " + e.getMessage());
110     }
111
112     desactivarEvento(num);
113
114     synchronized (this) {
115         finEventoMillis = 0;
116     }
117
118     if(num == 2){
119         LogHawkins.escribir("Eleven comienza a liberar niños de la Colmena");
120         mundo.revivirNiños();
121     }
122 }
123
124 public synchronized String getEventoActual() {
125     if (apagon) {
126         return "APAGÓN DEL LABORATORIO";
127     }
128     if (tormenta) {
129         return "TORMENTA DEL UPSIDE DOWN";
130     }
131     if (eleven) {
132         return "INTERVENCIÓN DE ELEVEN";
133     }
134     if (redMental) {
135         return "LA RED MENTAL";
136     }
137     return "SIN EVENTO";
138 }

```

```

140  public synchronized int getTiempoRestanteEvento() {
141      if (finEventoMillis == 0) {
142          return 0;
143      }
144
145      long restante = finEventoMillis - System.currentTimeMillis();
146
147      if (restante <= 0) {
148          return 0;
149      }
150
151      return (int) (restante / 1000);
152  }
153
154  public void run() {
155      while(true) {
156          try{
157              Thread.sleep((int) (Math.random()*30000)+30000);
158          }catch(Exception e){
159              System.err.println("Error: " + e.getMessage());
160          }
161
162          elegirEvento();
163      }
164  }
165  }

```

Demogorgons.java:

```

13 public class Demogorgons extends Thread {
14     private Mundo mundo;
15     private Eventos evento;
16     private String id;
17     private int zona=0;
18     private int zona_anterior=0;
19     private int capturas = 0;
20
21
22
23     public Demogorgons(Mundo mundo, Eventos eventos, int numid){
24         this.mundo = mundo;
25         this.evento = eventos;
26         this.id = String.format("D%04d", numid);
27         LogHawkins.escribir("Demogorgon " + id + " creado");
28     }
29
30     private void moverse(){
31         while(evento.hayEleven()){
32             try {
33                 esperar(500);
34             } catch (Exception e) {
35                 System.err.println("Error: " + e.getMessage());
36             }
37         }
38
39
40         zona_anterior = zona;
41         if(evento.hayRedMental()){
42             zona = mundo.zonaConMasNiños();
43             LogHawkins.escribir("Demogorgon " + id + " se dirige por RED MENTAL hacia " + zonaString(zona));
44
45         } else {
46             zona = (int) (Math.random()*4)+1;
47             while(zona_anterior == zona){
48                 zona = (int) (Math.random()*4)+1;
49             }
50         }
51         mundo.entrarDemogorgon(zona, this);
52         System.out.println("Demogorgon " + this.id + " ha entrado en " + zonaString(zona));
53
54         LogHawkins.escribir("Demogorgon " + id + " entra en " + zonaString(zona));
55     }

```

```

57 private String zonaString(int zona){
58     switch(zona){
59         case 0:
60             return "Sin zona";
61         case 1:
62             return "El Bosque";
63         case 2:
64             return "El Laboratorio";
65         case 3:
66             return "El Centro Comercial";
67         case 4:
68             return "El Alcantarillado";
69
70         default:
71             return null;
72     }
73 }
74
75 public synchronized void incrementar_capturas(){
76     capturas++;
77     LogHawkins.escribir("Demogorgon " + id + " aumenta sus capturas a " + capturas);
78 }
79
80 public String toString() {
81     return id;
82 }
83
84 public synchronized int getCapturas() {
85     return capturas;
86 }
87
88 private void esperar(int milisegundos) throws InterruptedException {
89     int tiempoTranscurrido = 0;
90
91     while (tiempoTranscurrido < milisegundos) {
92         Thread.sleep(100);
93         tiempoTranscurrido += 100;
94         mundo.comprobarPausa();
95     }
96 }
97
98 public void run(){
99     mundo.registrarHilo(this);
100     mundo.registrarDemogorgon(this);
101     System.out.println(this.id);
102     LogHawkins.escribir("Demogorgon " + id + " inicia ejecución");
103
104     while(true){
105         mundo.comprobarPausa();
106         while(evento.hayEleven()){
107             try {
108                 esperar(500);
109             } catch (Exception e) {
110                 System.err.println("Error: " + e.getMessage());
111             }
112         }
113
114         if (!evento.hayApagon()) {
115             if (zona != 0) {
116                 mundo.eliminarListaDemogorgon(zona, this);
117
118                 LogHawkins.escribir("Demogorgon " + id + " abandona " + zonaString(zona));
119             }
120
121             moverse();
122         }
123
124         mundo.demogorgonAtacar(zona, this);
125     }
126 }
127 }

```